Class 7

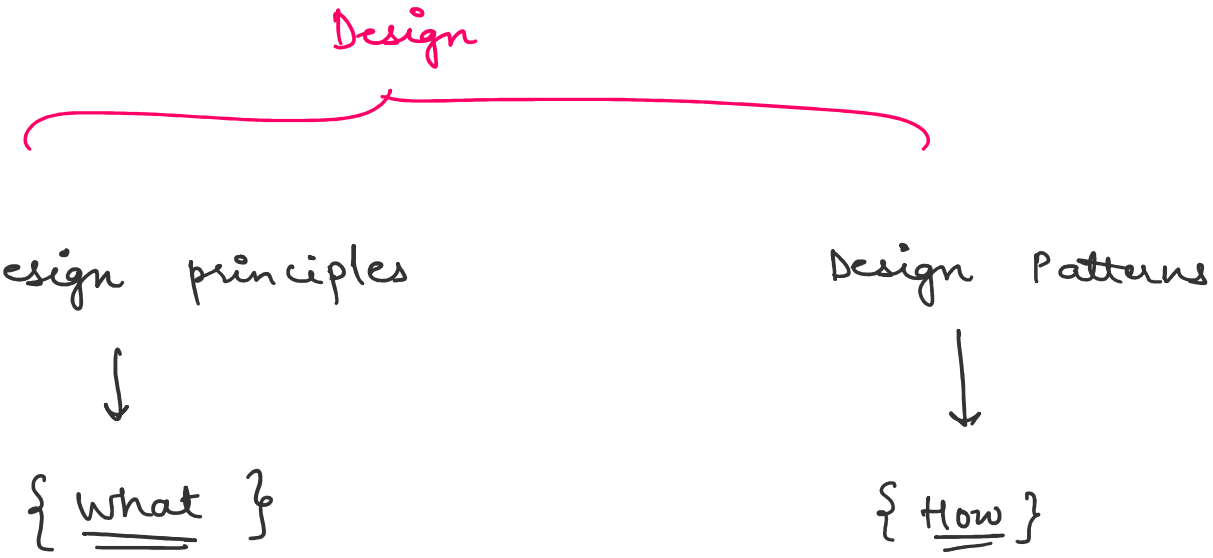
Wednesday, June 17, 2026 9:04 PM

Design principle

→ DRY - Do not repeat yourself

→ YAGNI - You aren't gonna need it.

→ SOLID



Design Patterns

↓

Implementation of design principles.

↓

GoF → Gang of Four

↓

Part 1 → OOPS, SOLID

Part 2 → 23 design pattern

Why learn Design patterns —

1. Shared vocabulary.
2. Saves time.
3. Better S/w design.

Types of Design Patterns

Creational DP	Structural DP	Behavioural DP
↓	↓	↓
Object creation	Class Structures	Class interaction

Components of DP

→ Name

→ Problem

→ Soln

→ Advantages

→ Disadvantages.

Creational DP –

Singleton DP

↓

creating single object

Party → Mic ↙ (single)

Why do we need singleton ?

→ Shared Resources.

↓

Logging

↓

Log file

unlock() →

↓

openMsg()

↓

selectSender()

↓

composeMsg()

↓

send()

→ Expensive object creation

↓

DB connⁿ

Request to DB server

↓

Auth process

↓

N/w connⁿ established

↓

Allocated resources

↓

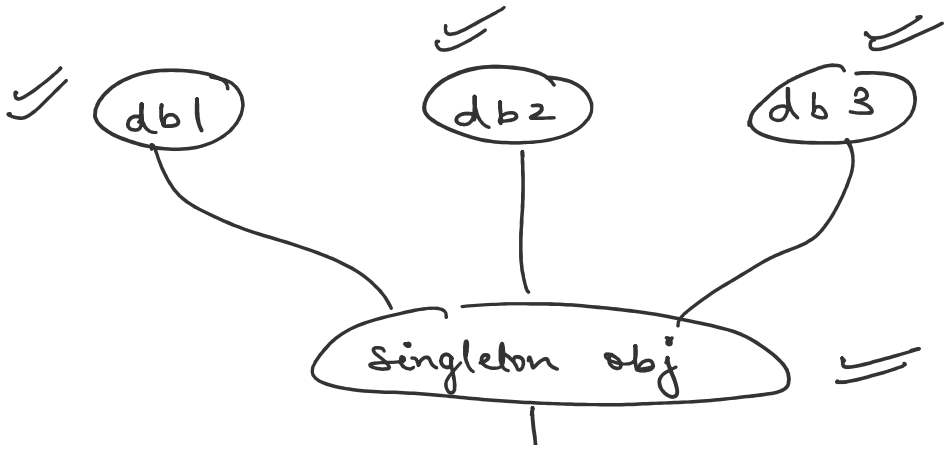
Creating sessions

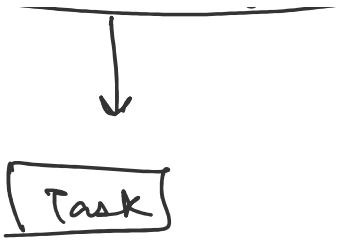
DB connⁿ

↓

Expensive opⁿ

```
DB db1 = new DB();
" db2 = " "
" d3 = " "
```






```
class A
{
    A()  →  constructor
                ↓
}           Multiple instances
```

Rule 1 – Private constructor

```
class A
{
    private A() { — }
          ↓
}
    Controlling object creation
```

Rule 2 - create private static instance variable

```
private static Singleton instance;
```

↓

controlling access

Rule 3 - public static method

```
public static Singleton getInstance()
{
    return instance;
}
```

↓

```
Singleton.getInstance();
```

↓

single object of singleton class.

Types of singleton -

→ Early loading | Eager loading

→ Lazy loading

→ Thread safe lazy loading

- synchronized
- Double checking

→ Enum singleton

Eager loading → wasting memory

References:

- Effective java
- Head first design patterns
- https://refactoring.guru/design-patterns

Design Principles

Design principles are guidelines, recommendations, and best practices for designing good software.

They define the characteristics that good software should possess:

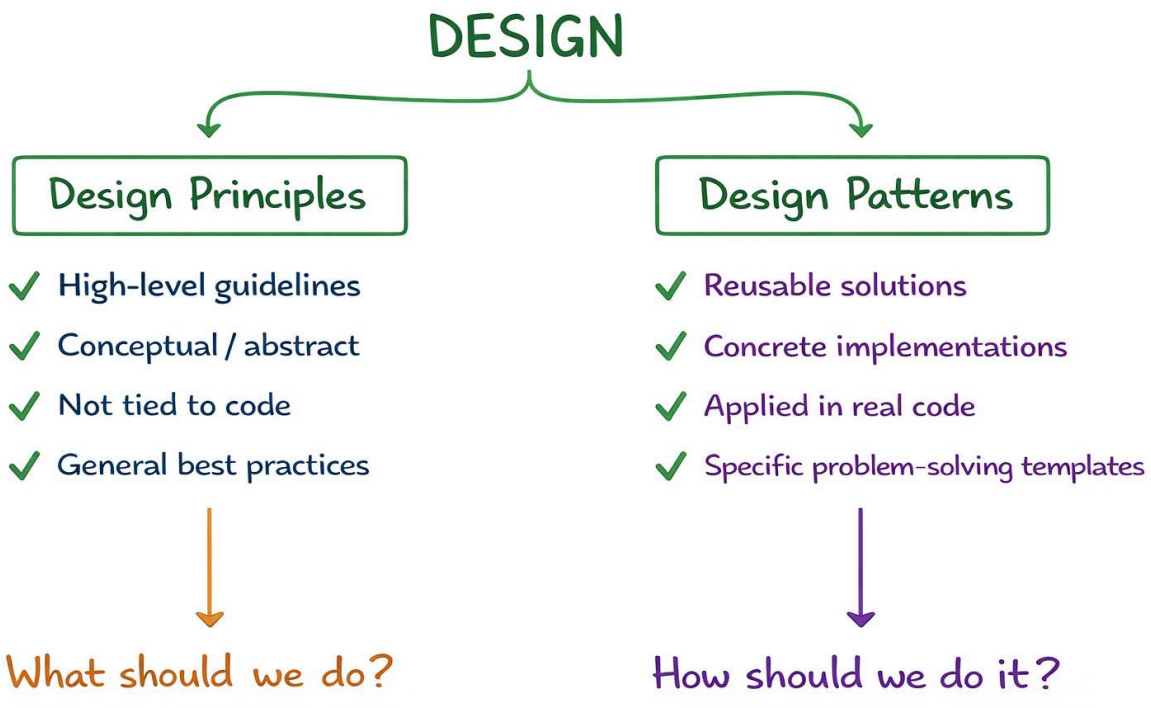
- Maintainability
- Scalability
- Flexibility
- Reusability
- Extensibility
- Low Coupling
- High Cohesion

Examples:

- SOLID Principles
- DRY (Don't Repeat Yourself)
- KISS (Keep It Simple, Stupid)
- YAGNI (You Aren't Gonna Need It)

Design principles tell us: **"What should a good software design look like?"**

They are not concrete solutions but general guidelines.



Design Patterns

Design Patterns are practical implementations of design principles. They are:

- Reusable solutions (blueprints) to common software design

problems
- Templates that guide software design
- Proven approaches used repeatedly in real-world applications

They are not code, but templates that guide how to design systems.

They define:
- Problem (when to use)
- Solution (how to solve)
- Consequences (pros/cons)

Design patterns tell us: **"How do we implement good design principles in practice?"**

Why Learn Design Patterns

1. Shared Vocabulary

Design patterns provide a common language for developers.

Instead of explaining: "We need a mechanism that creates objects without exposing creation logic."

We can simply say: "Let's use a Factory Pattern."

Developers immediately understand the design approach.

This improves communication during:
- Design discussions
- Code reviews
- Interviews

- Team collaboration

2. Saves Time

Most software projects encounter similar design problems.

Instead of solving the same problem repeatedly from scratch, we can use proven solutions that have already been tested and refined by experienced developers.

This reduces:

- Development effort
- Design mistakes
- Maintenance cost

3. Better Software Design

Design patterns help create software that is:

- Maintainable
- Reusable
- Extensible
- Flexible
- Scalable

They encourage good design practices and reduce tight coupling between components.

4. Interviews

Design patterns are commonly asked in:

- OOP Interviews
- LLD Interviews
- System Design Interviews
- Senior Developer Interviews

Understanding design patterns demonstrates:

- Strong OOP knowledge
- Design thinking
- Ability to solve real-world software problems

Important:

- Do not overuse patterns
- Use only when it simplifies the design

Gang of Four (GoF)

The most popular design patterns were documented in the book: *"Design Patterns: Elements of Reusable Object-Oriented Software".*

The authors are known as the **Gang of Four (GoF)**:

- Erich Gamma
- Richard Helm
- Ralph Johnson
- John Vlissides

The book contains:

- **Part 1**
  Discussion on:
  - Benefits of OOP
  - Challenges of OOP
  - Principles of reusable object-oriented design

- **Part 2**
  Description of:
  - 23 Classical Design Patterns

Their findings:

- All OOP applications face **repetitive problems**.
- Most software design problems can be categorized into three broad areas:
  1. Object Creation
  2. Object Structure
  3. Object Behaviour

Types of Design Patterns

1. Creational Patterns

Focus: Object Creation

Purpose: Provide flexible and controlled ways of creating objects.

Questions Answered

- How will an object be created?
- Where will an object be created?
- How many objects will be created?

- Who controls object creation?

Patterns:

- **Singleton** – Only one instance, global access. (Eager vs Lazy, Thread safety, vs Static class).
- **Factory** – Creates objects without exposing creation logic.
- **Factory Method** – Subclasses decide which object to create.
- **Abstract Factory** – Factory of factories; groups related factories.
- **Builder** – Step-by-step object construction.
- **Prototype** – Clone existing objects instead of creating new.
- **Object Pool** – Reuse expensive objects instead of recreating.

2. Structural Patterns (Class Structure)

Focus: How classes are structured and related

Purpose: Define how classes and objects are composed to form larger systems.

Questions Answered

- How should a class be structured?
- How should objects be connected?
- How should classes interact?
- How do we integrate third-party code?
- How can we reduce complexity in relationships?

Patterns:

- **Adapter** – Makes incompatible interfaces work together.
- **Bridge** – Decouple abstraction from implementation.
- **Composite** – Treat group of objects like a single object.
- **Decorator** – Add new behavior without modifying existing class.
- **Facade** – Provide a simple interface to a complex system.
- **Flyweight** – Reuse shared objects to save memory.

- **Proxy** – Control access to an object (lazy loading, security, logging).

3. Behavioral Patterns (Object Interaction)

Focus: How objects communicate and share responsibility

Purpose: Define how objects communicate and collaborate to accomplish tasks.

Questions:

- How do classes interact?
- How is behaviour implemented?
- How to make behaviour flexible?

Patterns:

- **Command** – Encapsulate actions as objects.
- **Interpreter** – Define grammar and interpret expressions.
- **Iterator** – Traverse collections without exposing internals.
- **Mediator** – Centralize complex communication between objects.
- **Memento** – Save and restore object state (undo functionality).
- **Observer** – One-to-many dependency (publish-subscribe).
- **State** – Change behavior based on internal state.
- **Strategy** – Select algorithms at runtime.
- **Template Method** – Define skeleton of algorithm, subclasses fill steps.
- **Visitor** – Add operations to objects without modifying them.
- **Chain of Responsibility** – Pass requests along handlers until processed.
- **Null Object** – Provide a default harmless behavior instead of null checks.

Components of a Design Pattern

Every design pattern typically consists of four parts:

1. Name
A meaningful identifier for the pattern.
Examples:

- Singleton
- Factory
- Observer

2. Problem
Describes:

- The scenario
- The recurring issue
- When the pattern should be used

3. Solution
Describes:

- Structure of the pattern
- Participating classes and objects
- Interaction between components

It provides the general design, not the exact implementation.

4. Consequences
Describes:

Advantages

- Flexibility
- Reusability

- Maintainability

Disadvantages

- Added complexity
- Additional abstractions
- Performance considerations

Pros and Cons of Design Patterns

Pros

- Reusable solutions
- Proven design approaches
- Better maintainability
- Improved scalability
- Easier communication among developers
- Encourages clean architecture
- Reduces design mistakes

Cons

- Can introduce unnecessary complexity
- May lead to overengineering
- Learning curve for beginners
- Not every problem requires a pattern
- Incorrect usage can make code harder to understand

When to Use

Before applying any pattern, ask:

1. Is this a recurring problem?
2. Does the pattern simplify the design?
3. Does it improve maintainability or flexibility?
4. Is a simpler solution available?
5. Are we solving a real problem or forcing a pattern?

Use design patterns only when they provide clear value.

Important Note

Design Patterns are tools, not rules.

The goal is not to use as many patterns as possible.

The goal is to:

- Solve problems effectively
- Improve software design
- Reduce complexity
- Make code easier to maintain

A simple solution is often better than an unnecessary design pattern.
Remember: Design Principles tell us what good design looks like, while Design Patterns show us how to achieve it in practice.

Singleton Design Pattern

Singleton is a creational design pattern that ensures:

- Only one instance of a class is created throughout the application.
- A global access point is provided to that instance.
- Every class that needs the object uses the same shared instance instead of creating a new one.

For a particular class:

- Only one object should ever exist.
- All other classes use the same shared object.
- No class is allowed to create its own copy.

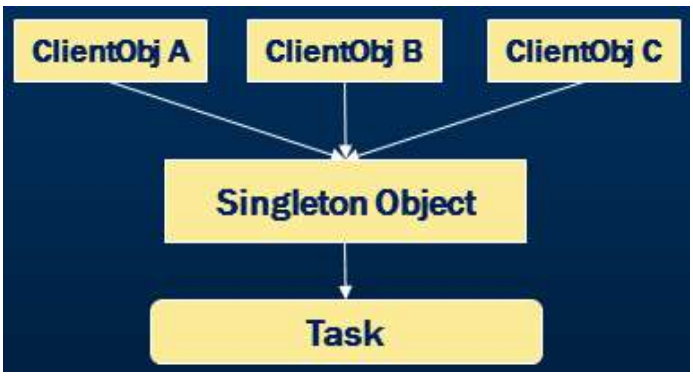
Instead of:

```
Database db1 = new Database();
Database db2 = new Database();
Database db3 = new Database();
```

Everyone uses:

```
Database db = Database.getInstance();
```

and receives the same object.



Why Do We Need Singleton?

i) Shared Resource

Sometimes multiple parts of the application need access to the same resource.

Examples:

- Logging system
- Configuration manager
- Database connection
- Cache
- Database connection pool
- Shopping cart/session data
- Printer spooling
- File handling

Creating multiple instances can lead to:

- Data inconsistency
- Memory waste
- Resource conflicts

ii) Expensive Object Creation

Some objects are expensive to create.

Example: Database Connection

Creating a database connection is not just object creation.

When a DB connection is created:

1. Request is sent to the database server.
2. Authentication happens.
3. A network connection is established.
4. Resources are allocated on the database server.
5. A session is created.

This is an expensive process.

Instead of creating multiple database connection manager objects, we create one object and share it across the application.

If every class creates its own database connection:

- **OrderService** -> creates DB connection
- **UserService** -> creates DB connection
- **PaymentService** -> creates DB connection

we waste:

- Memory
- CPU
- Network resources

Instead, all classes can share the same connection manager object.

iii) No Meaning of Multiple Instances

Some classes naturally should have only one instance.
Examples:

- Logger
- Configuration Manager
- Thread Pool manager

Having multiple objects provides no additional benefit.

Real-life Analogy

Imagine you're at a party, and there's only **one microphone**. Everyone who wants to speak must use that one mic — not bring their own. This

avoids noise and confusion.

In programming, this "**one mic**" is the **Singleton** object — used by everyone.

If multiple instances are created, they can cause **conflicts, unnecessary memory use, or inconsistent behavior**.

Logging Example
Imagine a big application with many classes (C1, C2, C3, etc.). Each class needs to write to the **same log file**, app.log.

If each class creates its own logger, you'll end up with scattered logs.

Instead, one **singleton logger object** ensures all logs are centralized in the same place.

To do this all classes use a single **Log object** and that Log object should be **one and only one** — no duplicates. Hence, it must be a **Singleton**.

Why Can't We Use Constructors?

A constructor:

- Always creates a new object.
- Never returns an old object.

Example:

```
Database db1 = new Database();
Database db2 = new Database();
```

This creates two different objects.

To prevent this:

- Constructor is made private.
- Object creation is controlled through getInstance().

Key Rules to Implement Singleton

Rule 1: Private Constructor

Prevents external object creation.

```
private Singleton() {
}
```

Rule 2: Private Static Instance Variable

Stores the only object.

```
private static Singleton instance;
```

Rule 3: Public Static Method

Provides global access to the object.

```
public static Singleton getInstance() {
    return instance;
}
```

Types of Singleton Implementations

- Eager Initialization/Early Loading
- Lazy Initialization/Loading (On demand loading)
- Thread Safe Lazy Initialization
  - synchronized
  - Double Checked Locking
- Enum Singleton

Eager Initialization

- Instance is created at **class loading time**, whether needed or not. It's **always ready**, but might **waste memory**.
- Simple but **not memory efficient** if instance is never used. The object is created before it's used.
- It's **thread-safe by default** — no need for extra coding to handle multiple threads.

Real-Life Analogy

Think of it like this: there's a party at 8:30 PM. But one guest arrives at 8 AM and says, “I’ll stay here all day.” This guest eats the food meant for the party and interrupts your preparations.

In programming, **eager loading** works the same way. An object (like a singleton) is created as soon as the program starts — even if it's not

needed yet. This takes up memory unnecessarily and could slow things down.

```
public class EagerSingleton {
    // Private constructor to prevent instantiation
    private EagerSingleton() {
        System.out.println("Singleton Instance Created");
    }

    private static EagerSingleton instance = new
EagerSingleton();

    public static EagerSingleton getInstance() {
        return instance;
    }

    public void showMessage(String message) {
        System.out.println("Show message method: " + message);
    }

    public static void printMessage(String message) {
        System.out.println("Print message method: " + message);
    }
}
```

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class Main {
    public static void main(String[] args) {
        EagerSingleton singleton1 = EagerSingleton.getInstance();
        EagerSingleton.printMessage("Static method call");
        singleton1.showMessage("Singleton 1");
```

```
        EagerSingleton singleton2 = EagerSingleton.getInstance();
        singleton2.showMessage("Singleton 2");


        // Thread safe
        ExecutorService exe = Executors.newCachedThreadPool();

        Runnable task1 = () -> {
            EagerSingleton singleton3 =
EagerSingleton.getInstance();
            singleton3.showMessage("Singleton 3");
        };

        Runnable task2 = () -> {
            EagerSingleton singleton4 =
EagerSingleton.getInstance();
            singleton4.showMessage("Singleton 4");
        };

        exe.submit(task1);
        exe.submit(task2);
        exe.shutdown();
    }
}

/* *
Output:
    Singleton Instance Created
    Print message method: Static method call
    Show message method: Singleton 1
    Show message method: Singleton 2
    Show message method: Singleton 4
    Show message method: Singleton 3
* */
```

Why Is It Thread Safe?

Static variables are initialized by JVM during class loading.
JVM guarantees:

- Class loading happens once.
- Static initialization happens once.

Therefore only one object is created.

Advantages

- Simple implementation
- Thread safe by default
- No synchronization required

Disadvantages

- Wastes memory if object is never used
- Resources are allocated early
- Cannot pass runtime parameters during creation